

Unit – 1 : Session – 6 : SLO – 2 | Modular Programming

Activity (Solutions)

- 1) Program broken into reusable methods: `static int readN(){ return new java.util.Scanner(System.in).nextInt(); }` `static boolean isEven(int n){ return n%2==0; }` `static void report(int n){ System.out.println(n + (isEven(n) ? " even" : " odd")); }`
- 2) Main + two helpers: `public class Main { static int square(int x){ return x*x; } static void printResult(int v){ System.out.println("Result: " + v); } public static void main(String[] a){ printResult(square(6)); } }`
Output: Result: 36.
- 3) Code reuse via static methods: static methods belong to the class, not instances, so any code can call `ClassName.method()` without creating objects — Java's `Math.sqrt()`, `Math.pow()`, `Integer.parseInt()` are examples; our own utility classes (e.g., a `Utils` class with static helpers) give the same reuse across the program.